

# 0.24. Les deux axes à la suite

`deplace_x` puis `deplace_y`, sans « = » : la lettre va à droite, puis descend depuis la colonne où elle est arrivée.

## 1. Ce qu'il faut comprendre

On garde la forme courte du 0.23, et on utilise **les deux axes** : d'abord `deplace_x`, puis `deplace_y`.

**Ce qui est nouveau ici** : deux variables, `x` et `y`, et chaque fonction reçoit **les deux**. `deplace_x(x, y, ...)` avance sur la ligne `y` ; `deplace_y(x, y, ...)` descend dans la colonne `x`.

**C'est pour cela qu'il faut garder `x` à jour** : `deplace_y` part de la colonne `x`, celle où `deplace_x` a laissé la lettre (5). Si `x` était resté à 0, la descente partirait de la colonne 0, et la lettre sauterait d'un coup.

**Les nombres**, en bas de l'écran (ligne 17) : `x` en colonne 0, `y` en colonne 4.

### Les fonctions de cette leçon — et le tuto de chacune

- `#include <deplace_x>` → [0.17.3. La fonction `deplace\_x\(\)` — avancer sur une ligne](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <nombre>` → [0.8.3. La fonction `nombre\(\)` — écrire un nombre](#)
- `#include <deplace_y>` → [0.20.3. La fonction `deplace\_y\(\)` — avancer sur une colonne](#)

## 2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1306 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      66 o ← la console, à l'allumage — la mise en route de la console
deplace_x()    158 o ← main() — une fonction fournie par le langage
deplace_y()    158 o ← main() — une fonction fournie par le langage
main()         69 o ← démarrage — ta fonction
VBlank         8 o ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage  28 o ← image() — attend l'image suivante
AttendreVBlank 16 o ← démarrage — attend que l'écran soit libre
AttendreAcces  12 o ← EffacerCases, EcrireNombre — attend de pouvoir écrire une case
AdresseCase    10 o ← poser(), effacer() — trouve l'adresse d'une case de l'écran
EffacerCases   12 o ← effacer() — efface des cases
EcrireNombre   33 o ← nombre() — écrit un nombre en chiffres
CopierTuiles   18 o ← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage — vide les deux cartes de l'écran
Tuiles        704 o ← CopierTuiles — la police entière
```

Lettres gravées : toutes (`poser()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `EcrireTexte`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

```
poser()      x 4    52 o sur place, dans deplace_x(), deplace_y() + AdresseCase (10 o)
effacer()    x 2    30 o sur place, dans deplace_x(), deplace_y() + AdresseCase (10 o), EffacerCases (12 o)
```

```
nombre()    x 2    20 o sur place, dans main() + EcrireNombre (33 o)
image()     x 3     9 o sur place, dans deplace_x(), deplace_y(), main() + AttendreImage (28 o)
Tes fonctions appelées :
deplace_x() x 1    19 o sur place, dans main()
deplace_y() x 1    19 o sur place, dans main()
```

## 1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <deplace_x> // fait avancer une tuile sur sa ligne
#include <ALPHABET>  // les lettres de la police : ALPHABET[0] est le A
#include <nombre>    // écrit un nombre en chiffres
#include <deplace_y> // fait avancer une tuile sur sa colonne
```

## 2. UNE VARIABLE — PAS

```
uint8_t pas = 5; // 5 cases, sur chaque axe
```

## 3. UNE VARIABLE — X

```
uint8_t x = 0; // la COLONNE de la lettre (axe X)
```

## 4. UNE VARIABLE — Y

```
uint8_t y = 0; // la LIGNE de la lettre (axe Y)
```

## 5. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    // 1. À droite : deplace_x reçoit x ET y — elle avance sur la ligne y (0).
    deplace_x(x, y, ALPHABET[0], pas); // x vaut 5
    nombre(0, 17, x);                  // en bas à gauche : 005

    // 2. En bas : deplace_y part de la colonne x - 5, là où la lettre est.
    //
    //   deplace_y(x, y, ALPHABET[0], pas);
    //           | |
    //           | +- y : la ligne de départ (0) ; la console y range l'arrivée (5)
    //           +---- x : la colonne, 5 grâce à la ligne d'avant
    deplace_y(x, y, ALPHABET[0], pas); // y vaut 5
    nombre(4, 17, y);                  // à côté : 005

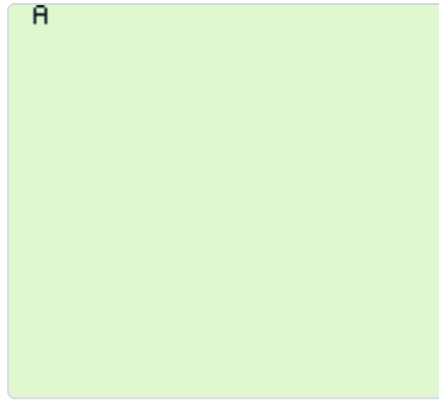
    while (true) {
        image(); // la lettre est en (5, 5)
    }
}
```

# 3. Ce qui se passe, étape par étape

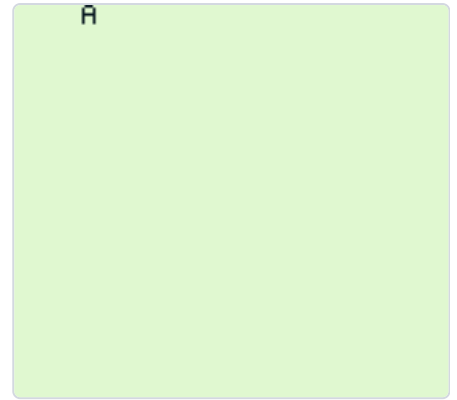
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



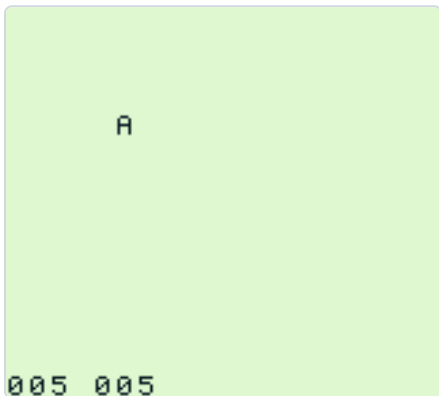
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180

## 4. Ce qu'on doit voir

Un A qui va de 5 cases à droite, puis descend de 5 lignes. En bas, 005 et 005.

## 5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ à droite d'abord, jusqu'en (5, 0)
- ✓ puis en bas depuis la colonne 5 : (5, 3)
- ✓ x et y valent 5
- ✓ 005 et 005 affichés (« 005 005 »)

Ensuite : **0.24.1. Les deux axes à la suite — de base, ailleurs** — chapitre 0 : Avant tout. · [gameboy3](#) — écrire une cartouche Game Boy en C++